

[← Go Back](#)

Hardware

GIGA Display Shield



Tutorials

[Getting Started with the GIGA Display Shield](#)[GIGA Display Shield GFX Guide](#)[GIGA Display Shield LVGL Guide](#)[GIGA Display Shield Draw Images Guide](#)[GIGA Display Shield Touch Interface](#)[GIGA Display Shield Automatic Screen Orientation with IMU](#)[GIGA Display Shield Microphone Guide](#)[GIGA Display Shield Camera Connector Guide](#)[GIGA Display Shield 3D Engine Tutorial](#)[SquareLine Studio With the GIGA Display Shield](#)[Display Camera Feed on GIGA Display Shield with OpenMV](#)[GIGA Display Shield emWin Guide](#)[GIGA Display Shield AppWizard Guide](#)[GIGA Display Shield Display Image From USB](#)[Home](#) / [Hardware](#) / [GIGA Display Shield](#)/ [GIGA Display Shield Draw Images](#)[Guide](#)

GIGA Display Shield Draw Images Guide

Learn how to use basic draw functions to create and display images on the GIGA Display Shield.

Author · Benjamin Dannegård · Last revision · 12/05/2024

The most basic use of the GIGA Display Shield is to draw an image on the screen using code. This is made easy by using the library `ArduinoGraphics`. In this tutorial we will go through how to draw the Arduino logo on the GIGA Display Shield with the commands provided by this library.

This is a great tutorial for getting started with your shield and exploring what possibilities the library gives us.

Hardware & Software Needed

[Arduino GIGA R1 WiFi](#)[Arduino GIGA Display Shield](#)[Arduino IDE](#)

ON THIS PAGE

Hardware & Software Needed

[Downloading the Library and Board Package](#)[Using Draw Feature in a Sketch](#)[Displaying Images](#) +[Conclusion](#)

Downloading the Library and Board Package

[Help](#)

Make sure the latest GIGA Board Package is installed in the Arduino IDE. **Tools > Board > Board Manager....** Here you need to look for the **Arduino Mbed OS Giga Boards** and install it, the [Arduino_H7_Video library](#) is included in the Board Package and is needed for the examples to work. Now you have to install the library needed for the graphical display features. To do this, go to **Tools > Manage libraries...**, search for **ArduinoGraphics**, and install it.

For more information about libraries and how to install them with the IDE, visit our [libraries tutorial](#).

Using Draw Feature in a Sketch

First, we need to include the library that we will be using and define the display screen:

```
1 #include "Arduino_H7_Vide
2 #include "ArduinoGraphics
3
4 Arduino_H7_Video Display(
```

As we only want to draw something on the screen once, we can put all the drawing code in the `void setup()`. If we instead put this code in the `void loop()` function it will keep drawing the given image over and over.

Let's first draw the background of the image. Start by initializing the display class from the library with

drawing things on the screen we need to use `Display.beginDraw()`. Now the background color can be set with

```
Display.background(255, 255, 255)
```

, where RGB values are entered within the parentheses.

`Display.clear()` will clear anything that has been drawn on the screen, but it will leave the background as it was set before.

```
1 void setup() {  
2   delay(3000);  
3   Display.begin();  
4  
5   Display.beginDraw();  
6   Display.background(255,  
7   Display.clear());
```

Next, let's draw the circle that will be the base of the logo. First, set the color of the circle with

`Display.fill(0x008184);`. Then draw the circle with the command

`Display.circle()`. Inside the parentheses enter the x position, y position, and diameter of the circle. We can make the positioning easy by using the display's total width and height as a base for our position measurements.

```
1 Display.fill(0x008184);  
2 Display.circle(Display.
```

Next, we want to draw the two circles that are in the bigger circle of the Arduino logo. First, we will set the color of the circles we are

lines we are drawing use

`Display.stroke()`, where the color of the stroke will be the RGB value entered into the parentheses, in this case they will be white. Then we use `Display.noFill()` to indicate that we don't want to fill the circles we are drawing. Lastly, to get the thickness of the circles we can use a simple `for` loop that will draw circles in a slightly changed position 30 times, to give the appearance of the thick circles in the logo.

```
1 Display.stroke(255, 255, 255);
2 Display.noFill();
3 for (int i=0; i<30; i++)
4   Display.circle((Display.width/2), (Display.height/2), radius);
5   Display.circle((Display.width/2), (Display.height/2), radius);
6 }
```

Lastly, let's draw the plus and minus symbols inside the two circles of the logo. The easiest way to draw these is to use the `Display.rect()` function, which will draw a rectangle with the parameters given. Let's first give the color with `Display.fill()` as we have done before. When using `Display.rect()` we can make it easier and use the display's dimensions as the base of the sizes. After entering the width and height of the rectangle we can enter:

```
1 Display.fill(255, 255, 255);
2 Display.rect((Display.width/2), (Display.height/2), width, height);
3 Display.fill(255, 255, 255);
4 Display.rect((Display.width/2), (Display.height/2), width, height);
5 Display.fill(255, 255, 255);
6 Display.rect((Display.width/2), (Display.height/2), width, height);
```

Now that the drawing is done the `Display.endDraw()` function can be called.

```
1 Display.endDraw();  
2 }
```

The complete code can be found as an example in the **Arduino_H7_video** library, it is called **ArduinoLogoDrawing**. The full sketch is also below. Now upload the entire sketch and you should see the Arduino logo being drawn on the display.

Full Sketch

```
1 #include "Arduino_H7_Vi
2 #include "ArduinoGraphi
3
4 Arduino_H7_Video Displa
5 //Arduino_H7_Video Disp
6
7 void setup() {
8     delay(3000);
9     Display.begin();
10
11     Display.beginDraw();
12     Display.background(25
13     Display.clear();
14     Display.fill(0x008184
15     Display.circle(Displa
16     Display.stroke(255, 2
17     Display.noFill();
18     for (int i=0; i<30; i
19         Display.circle((Dis
20         Display.circle((Dis
21     }
22     Display.fill(255, 255
23     Display.rect((Display
24     Display.fill(255, 255
25     Display.rect((Display
26     Display.fill(255, 255
27     Display.rect((Display
28     Display.endDraw();
```

Testing It Out

Now that it is all uploaded your GIGA Display Shield should look like the image below:



Sketch running on the
GIGA Display Shield

Disnlaving Images

Now let's have a look at how we can use the **ArduinoGraphics** library to display images on the GIGA Display Shield.

Converting an Image

Using an online image converter you can pick any image you would like to be displayed on the display shield. However keep in mind the display is 480x800 in size. The format of the converted image needs to be Binary RGB565. Using an online image converter like the [LVGL image converter tool](#) will let you pick this as an option. Simply pick the "Binary RGB565" option under "Output format", your desired color format and hit "Convert". You will now have an image that is ready for use in an Arduino sketch.

Displaying the Image on the Display

We will be using the example sketch "ArduinoLogo" as the basis for the sketch that lets us display an image. The example sketch can be found under **File > Examples > Arduino_H7_Video > ArduinoLogo**.

Running the example sketch as is will display the Arduino logo on the screen, like in the image below:



Arduino Logo on the GIGA Display Shield

Now to use the image that we converted in the last step. Use the macro inside the example sketch. This makes use of the `incbin.h` translation library. The necessary files are located in the folder for the example sketch.

At the start of the sketch you can see these lines commented out:

```
1  /*
2  #define INCBIN_PREFIX
3  #include "incbin.h"
4  INCBIN(test, "/home/user/
5  */
```

Uncomment these lines, and change the path to the image to the correct one. For Mac and Linux users the syntax of the path is correct as

```
"/home/user/Downloads/test.bin"
```

. For Windows users the path needs to be an absolute path, like this:

```
"C:\\USERNAME\\Downloads\\test.bin"
```

. Now we need to change the `Image` variable to use our image.

By default the image we import will be called `test`. The line

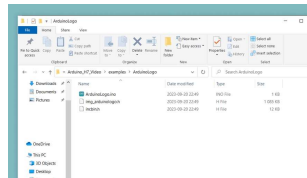
```
Image
img_arduinologo(ENCODING_RGB16,
(uint8_t *) texture_raw, 300,
300);
```

needs to have one argument changed, `texture_raw` should now be `testData`. So the line should be

```
Image
img_arduinologo(ENCODING_RGB16,
(uint8_t *) testData, 300,
300);
```

Full Sketch

To run this sketch you will need to have the image file that is used in the code in the correct location. In the example from the library, the image is in the same folder as the sketch, as shown in the image below. Please make sure the image you want to use matches the location indicated in the code.



Folder structure for the sketch

```
1  /*
2    ArduinoLogo
3
4    created 17 Apr 2023
5    by Leonardo Cavagnis
6  */
7
8  #include "Arduino_H7_Video_Display.h"
9  #include "ArduinoGraphics.h"
10
11 #include "img_arduino_logo.png"
12 // Alternatively, any raw image file can be used
13 // Online image converter: https://www.online-image-converter.com/
14 /*
15 #define INCBIN_PREFIX
16 #include "incbin.h"
17 INCBIN(test, "/home/user/ArduinoLogo/img_arduino_logo.png")
18 */
19
20 Arduino_H7_Video_Display display = Arduino_H7_Video_Display(1, 1);
21 //Arduino_H7_Video_Display display = Arduino_H7_Video_Display(1, 1);
22
23 Image img_arduino_logo = Image(img_arduino_logo.png);
24
25 void setup() {
26   delay(3000);
27   display.begin();
28 }
```

Conclusion

In this tutorial, we used basic drawing functions with the GIGA Display Shield. Using the `ArduinoGraphics` library we managed to draw the Arduino logo with just a few simple commands. Using these basic functions it is possible to create most images that you can think of. Now you can let your imagination run wild and draw to your heart's content!

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

